

What If You Could Pause a Language Model While It Is Learning?

Large language models are often taught in one of two ways. Learners either see diagrams and equations explaining attention, embeddings and loss, or they are handed a notebook and asked to run code.

There is a gap between understanding the theory and developing a feel for what a model is doing. A learner may know that loss should fall, but not recognise what the model has learned at different points in training. They may know that temperature affects generation, but never compare outputs from the same checkpoint. Before the experiment starts, they may also have to deal with Python environments, GPU drivers and cloud infrastructure.

LLM Experiments Lab is a browser-based laboratory designed to close that gap. It exposes the controls that help people learn while hiding setup work that is not central to the experiment. The project was carried out under the Nebius Serverless AI Challenge and was inspired by the Nebius Academy AI Performance Engineering course, a demanding 14 week programme which I recently completed.

The problem: notebooks expose everything at once

Jupyter and Google Colab are excellent tools for practitioners who already know which code and parameters they want to change. For a beginner, however, the freedom of a notebook can become noise.

Which settings matter? Which variables should remain fixed? Did the model fail because the learning rate was too high, because the architecture was wrong, or because a code cell was run out of order?

The Lab replaces the blank canvas with controlled experiments. A user selects a preset, changes parameters such as layer count, attention heads, embedding size, learning rate and batch size, and starts training from the browser. Current presets include a Tiny Transformer, an MoE variant of the latter, a deliberately high learning rate and an LSTM comparison.

The application is not intended to replace notebooks. It creates a guided step between theory and unrestricted code.

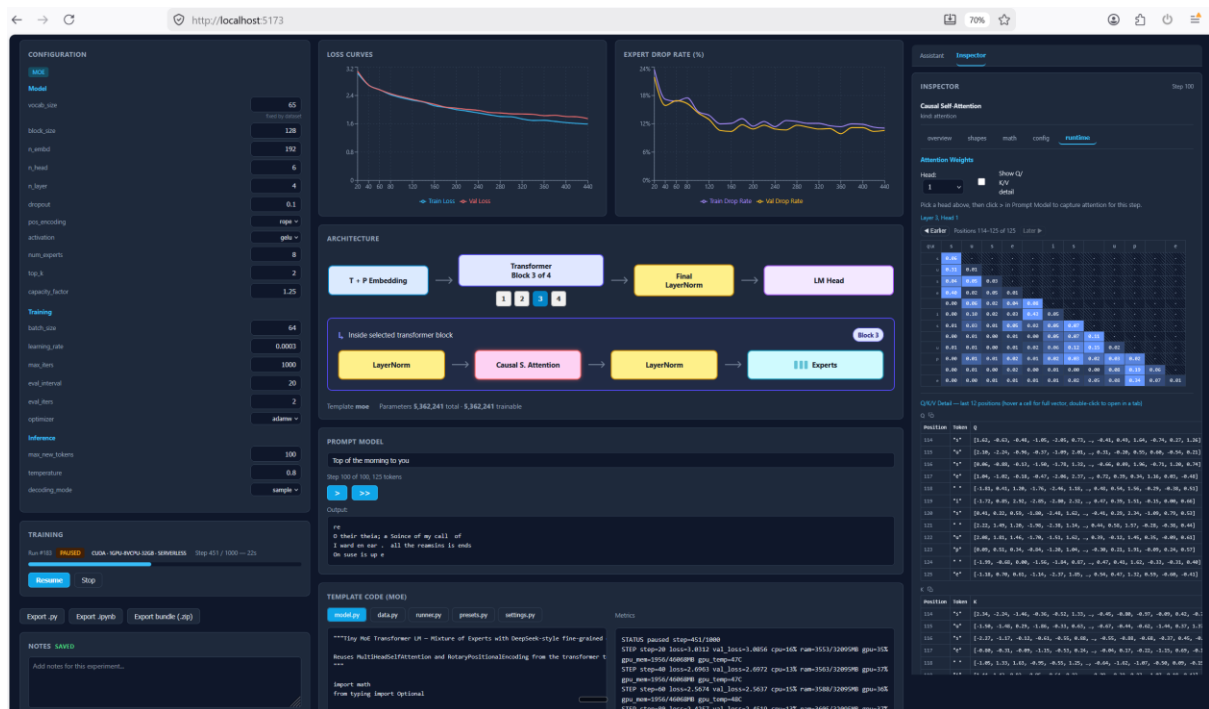


Figure 1: React/Vite Dashboard showing config, training curves, prompt box, and attention maps

Architecture

The system separates the browser interface, the experiment controller and the compute worker. Local training remains available for development, while the same FastAPI controller can route a run to a Nebius Serverless AI endpoint.

Architecture and lifecycle diagrams are shown below

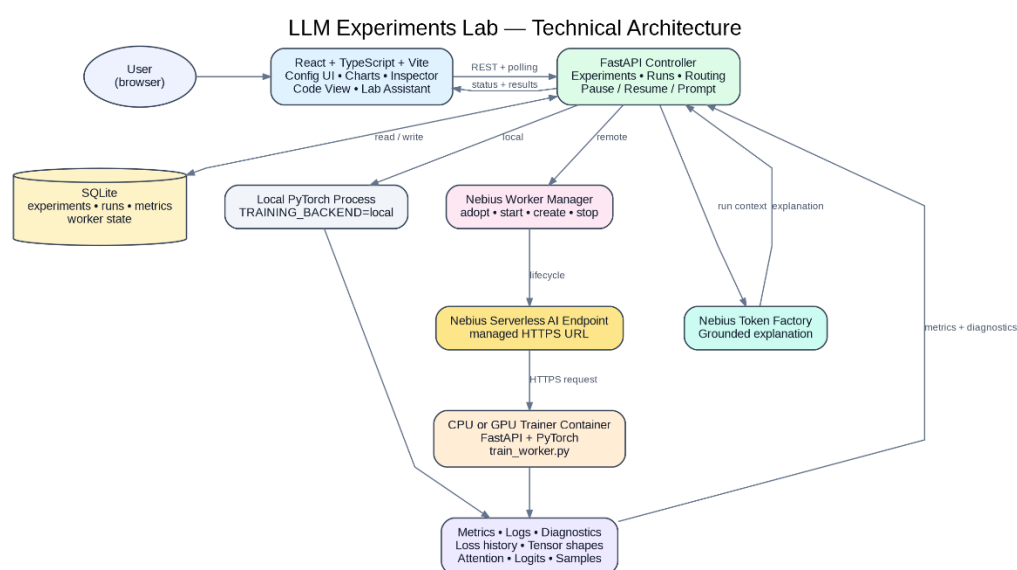


Figure 2: Technical Architecture

Remote Training Lifecycle on Nebius (Simplified)

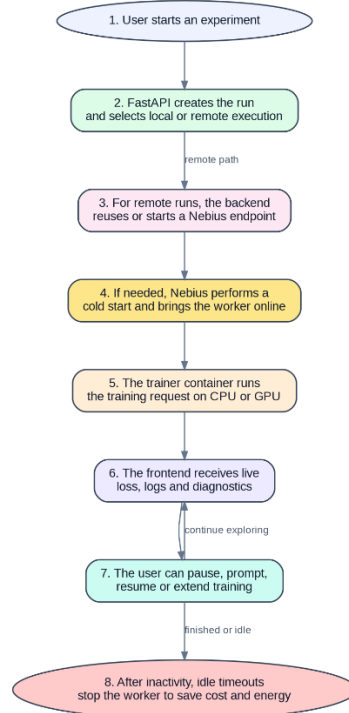


Figure 3: Training Lifecycle

The React/Vite frontend contains the configuration controls, training chart, model Inspector, code view and Lab Assistant. It calls a FastAPI backend that owns experiments and training runs. SQLite stores experiment configurations, run metadata, metrics and worker state.

The backend can execute locally or use `nebius_endpoint`. This switch allows the same browser workflow to work on a laptop during development and on remote CPU or GPU infrastructure when more compute is required.

Implementation

When the user presses **Start**, FastAPI creates a run and selects the execution backend. In local mode, the runner starts a PyTorch training process. In remote mode, a worker manager checks whether the appropriate Nebius endpoint already exists.

The endpoint lifecycle is deliberately idempotent. The manager first looks for a running endpoint it can adopt. If it finds a stopped endpoint, it starts it. It creates a new endpoint only when neither is available.

It also verifies that an endpoint recorded as ready in SQLite is genuinely running, because an endpoint might have been deleted manually in the Nebius console. A debounce prevents two simultaneous requests from provisioning duplicate workers.

CPU and GPU workers use separate Docker images. The CPU image installs CPU-only PyTorch rather than downloading CUDA dependencies it cannot use. Both containers expose port 8000 and run the FastAPI application. The training route launches the repository's `train_worker.py` process and the selected model template.

Interactive control was an important architectural constraint. Nebius Jobs are well suited to background tasks, but they do not provide a public request URL. That makes them awkward for a lab that must accept pause, resume and prompt commands during training.

Serverless endpoints provide managed HTTPS access and can be stopped and restarted, so the project uses endpoints for interactive training. Longer, non-interactive workloads could still be moved to Jobs later.

The configured idle policy is ten minutes for a GPU worker and thirty minutes for a CPU worker. The frontend warns the user before shutdown and can send an activity heartbeat when they continue the session. This reduces the risk of leaving paid compute running after an experiment has been abandoned.

Training metrics are returned to the frontend and plotted while the run is active. Polling every two seconds was chosen as the straightforward first implementation, although a WebSocket route is also available.

The pause-and-prompt workflow lets the learner freeze training, generate text from the current checkpoint and then resume. They can also extend the iteration budget without discarding the run.

Looking inside a forward pass

Diagnostic hooks collect runtime tensor shapes, embeddings, attention data, query, key and value vectors, logits and leading token probabilities.

The language-model head shows the highest-ranking tokens, their logits and probabilities, and the token that was selected. During sampling, the selected token will not always be the one with the highest probability.

This makes it possible to inspect one forward pass and connect an architecture diagram to real values produced by the model. Learners can see how an input is represented, how attention transforms it and how the final vector becomes a probability distribution over the vocabulary.

Inference controls can also be changed without retraining. The user can switch between greedy decoding and sampling, change the temperature and adjust the maximum

output length. This demonstrates the distinction between what a model learned during training and how its output is selected during inference.

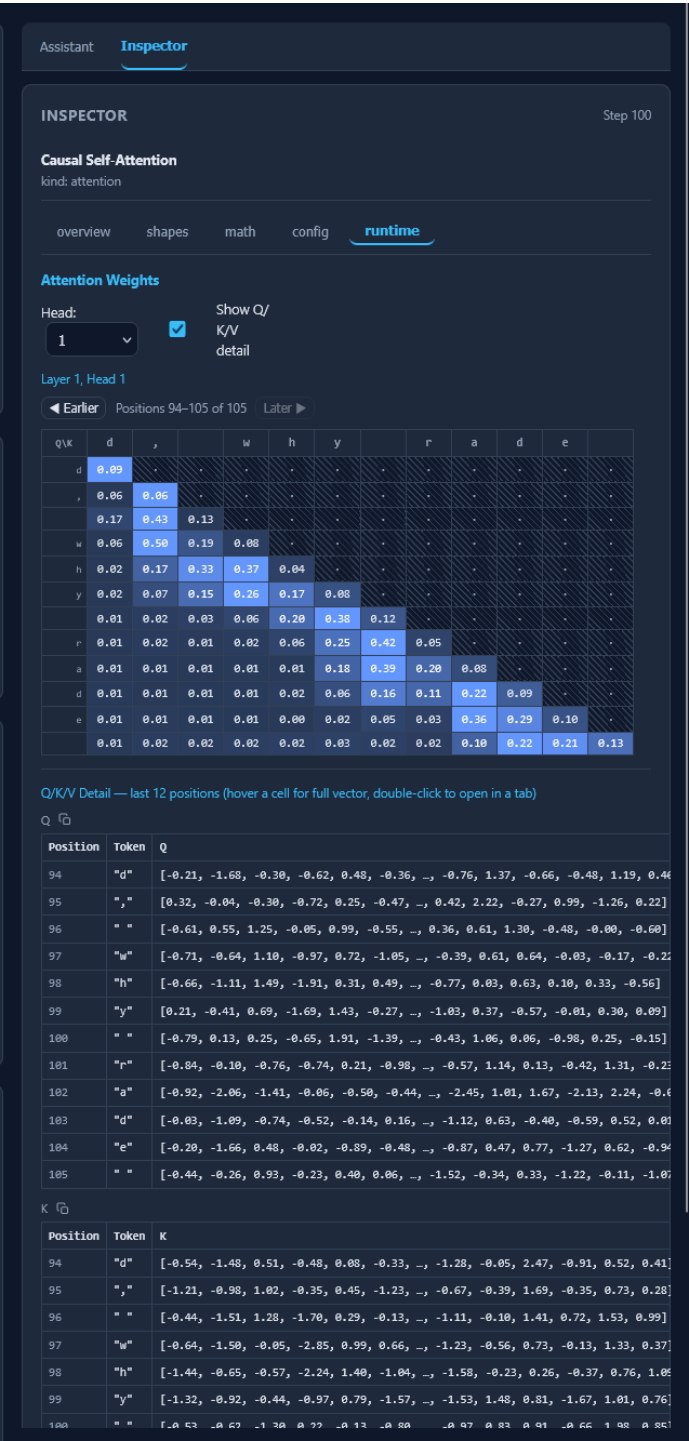


Figure 4: Attention map and Q/K/V vectors

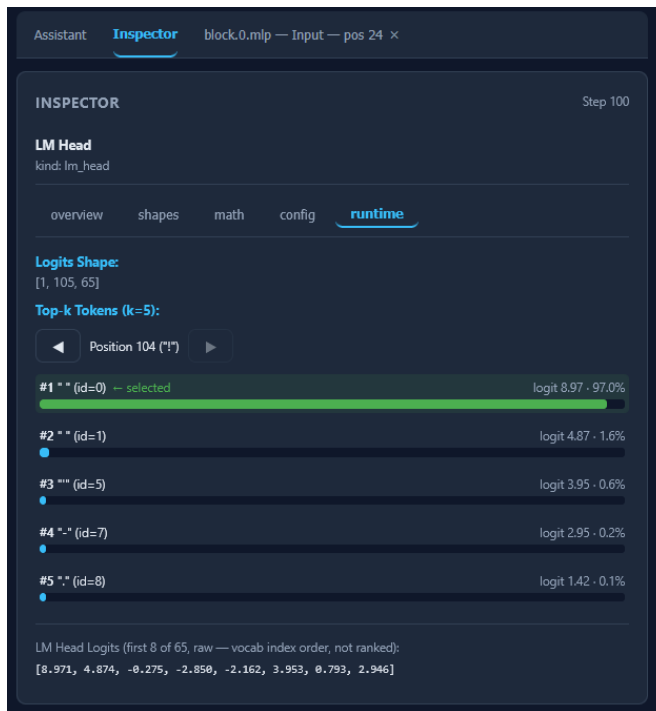


Figure 5: Top-k logits at LM Head

A grounded Lab Assistant

The Lab Assistant receives selected experiment context rather than acting as a generic chatbot. Its context can include the model configuration, loss history, recent training events, generated outputs and Inspector diagnostics.

Allowlisted tools can retrieve specific run metrics or experiment files without providing arbitrary access to the filesystem. Nebius Token Factory then generates an explanation grounded in the active experiment.

The assistant can therefore address questions such as why the loss increased after a configuration change, why a sampled token was selected despite not ranking first, or what a particular tensor represents in the forward pass.

Results and lessons

The local application supports preset selection, configuration editing, CPU or GPU training, live loss charts, pause, resume, stop, mid-training prompts, code inspection and export to Python or Jupyter.

The implementation exposed several practical lessons. Endpoint creation can take minutes, so lifecycle state must be visible to the user. A timed-out client command does not necessarily mean that an endpoint failed; it may still be starting. The manager therefore checks the live Nebius state before creating a replacement.

Separating the CPU and GPU images also prevents the CPU deployment from carrying an unnecessary CUDA dependency tree.

The current design uses one shared worker per device type, so a production version would need queuing or additional endpoints to support concurrent users.

Even at this stage, the core result is clear: learners can change a model, train it, interrupt it, inspect its internal calculations and ask questions grounded in the run—all through one browser interface.

The best way to understand a model is not only to read about it or call its API. It is to change something, predict what will happen, run the experiment and look inside.

#NebiusServerlessChallenge

Github repo: https://github.com/NnamdiOdozi/llm_experiments_lab/